COLLEGE OF COMPUTING AND INFORMATICS
PUTRAJAYA CAMPUS

LAB NO. 6:
Controlled Buffer Overflow Attack

SEMESTER 1 2026/2027

PROGRAMME : Bachelor of Computer Science (Cyber Security) (Hons.)
: Bachelor of Software Engineering (Hons.)

SUBJECT CODE : CCSB5113

SUBJECT : SOFTWARE SECURITY

DATE : Week 10 (26 Oct – 30 Oct)

DUE DATE : 3 Nov 2026 (1159PM)

NAME OF GROUP : MARVELLOUS GROUP

NAME OF MEMBERS :

No.	Full Name	ID Student	Class Group	Task
1.	Khairunnisa bt Osman	12345678	01B	Q1

INSTRUCTIONS TO STUDENTS:

1. Answer **ALL** questions.

2. All answers must be submitted via Brighten.

LAB NO. 6:
Controlled Buffer Overflow Attack

1. Objective

This lab demonstrates a controlled buffer overflow crash safely *(due to education purposes only).* Students will observe how writing beyond a buffer causes a segmentation fault without executing malicious code.

2. Requirements

- Ubuntu or WSL
- gcc installed (sudo apt install gcc)
- Text editor (nano, vim, etc.)

BUFFER OVERFLOW LEVEL 1

1. Run WSL

Since the WSL has been installed in the previous Lab 5, open WSL correctly by running wsl in PowerShell.

The output:

```
PS C:\Users\khairunnisa.osman> wsl
khairunnisaosman@g10-khairun:/mnt/c/Users/khairunnisa.osman$
```

This means you are inside Ubuntu.

Question 1: What is WSL?

2. Install gcc (in WSL Ubuntu)

Run this inside your Ubuntu terminal: sudo apt update
Then: sudo apt install gcc

It will ask for your password, type it *(the screen will not show your typing).*
Press *Enter*.

This will install:

- gcc
- libc
- essential C compiling tools

3. Confirm gcc is installed

Run: gcc --version

Question 2: What is GCC?

4. Create folder

```
mkdir bufferlab
cd bufferlab
```

Question 3: Function of mkdir?

5. Create (or open) a file with nano to insert vulnerable coding

Run: nano overflow.c

Paste the following code:

```
#include <stdio.h>
#include <string.h>

int main() {
   char buffer[8];
   printf("Enter something: ");
   gets(buffer);   // ❌ vulnerable function
   printf("You entered: %s\n", buffer);
   return 0;
}
```

CONTROL + *X* and *Enter* to save the file as per suggested name.

Question 4: What is nano?

6. Compile your vulnerable code and create executable binary file named as overflow:

```
gcc -fno-stack-protector -z execstack overflow.c -o overflow
```

You should receive the warning from GCC related gets function is dangerous.

Question 5: Screenshot your output

Question 6: What is the dangerous function in that coding?

5. Run the program (means you ignore the warning)

Run: ./overflow

6. Cause a buffer overflow 

Your buffer size = 8 bytes: *char buffer[8];*

To cause buffer overflow, enter a long string (more than 8 character), e.g.:
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

Expected result:

- **Segmentation fault (core dumped)**
- OR program crash
- OR unexpected behavior

Question 7: Screenshot your output

Question 8: What is the meaning of *"Segmentation fault (core dumped)"*?

Question 9: Which category of exception or error does it belong to: checked, unchecked, or fatal? Explain why.

Question 10: When you received the warning message by GCC about gets function
a) What type of message is this?
b) According to the concept of error message, which category of exception or error does it belong to: checked, unchecked, or fatal? Explain why."

7. Safe version to prevent buffer overflow

Run: nano fgetsoverflow.c

Paste the following code:

```
#include <stdio.h>
#include <string.h>

int main() {
   char buffer[8];
   printf("Enter something: ");

   // Use fgets instead of gets — SAFE
   fgets(buffer, sizeof(buffer), stdin);

   // Remove trailing newline (optional)
   buffer[strcspn(buffer, "\n")] = '\0';

   printf("You entered: %s\n", buffer);
```

```
    return 0;
}
```

Repeat the same save technique as previously.

8. **Compile your safe code and create executable binary file named as overflow:**

   gcc -fno-stack-protector -z execstack fgetsoverflow.c -o fgetsoverflow

9. **Run the program**

   Run: ./fgetsoverflow

   Enter a long string (more than 8 character), e.g.:
   AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

Question 11: Screenshot the output

Question 12: Explain why the system accepted AAAAAA only?

Question 13: What is the safe function been used in the coding?

BUFFER OVERFLOW LEVEL 2

1) Create new nano file
Run: nano overflow_crash.c

Paste the following code:

```
#include <stdio.h>
#include <string.h>

void crash_demo() {
   char buffer[10];
   char input[] = "THIS_INPUT_IS_TOO_LONG";

   printf("Buffer size      : %lu bytes\n", sizeof(buffer));
   printf("Input size       : %lu bytes\n", strlen(input));
   printf("Copying (unsafe)… This will crash.\n");

   for (int i = 0; i < strlen(input); i++) {
      buffer[i] = input[i];
   }

   printf("You will NOT see this message (program will crash first)\n");
}

int main() {
   crash_demo();
   return 0;
}
```

Repeat the same save technique.

2) Compile the Program
Run: gcc -fno-stack-protector -z execstack overflow_crash.c -o overflow_crash

3) Run the Program
Execute: ./overflow_crash

Expected output:
Buffer size: 10 bytes
Input size: 22 bytes
Copying (unsafe)… This will crash.
Segmentation fault (core dumped)

Question 1: Screenshot the output

Question 2: Explain what has happened.

4) Safe version to prevent buffer overflow
Run: nano overflow_safe.c

Paste the following code:

```
#include <stdio.h>
#include <string.h>

void safe_demo() {
   char buffer[10];
   char input[] = "THIS_INPUT_IS_TOO_LONG";

   printf("Buffer size      : %lu bytes\n", sizeof(buffer));
   printf("Input size       : %lu bytes\n", strlen(input));
   printf("Copying (safe)... Input will be truncated.\n");

   // Safe copy — only copy up to buffer-1
   strncpy(buffer, input, sizeof(buffer) - 1);

   // Force null termination
   buffer[sizeof(buffer) - 1] = '\0';

   printf("Safe copied buffer: %s\n", buffer);
   printf("Program continues normally (no crash).\n");
}

int main() {
   safe_demo();
   return 0;
}
```

5) Compile the Program
Run: gcc -fno-stack-protector -z execstack overflow_safe.c -o overflow_safe

6) Run the Program
Execute: ./overflow_safe

Question 3: Screenshot the output

Question 4: Explain why the system accepted THIS_INPU and says there was no crash?

8. Submission Requirements

Students must submit:

Buffer Overflow Level 1: Answers from 13 questions

Buffer Overflow Level 2: Answers from 4 questions